Template Method

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

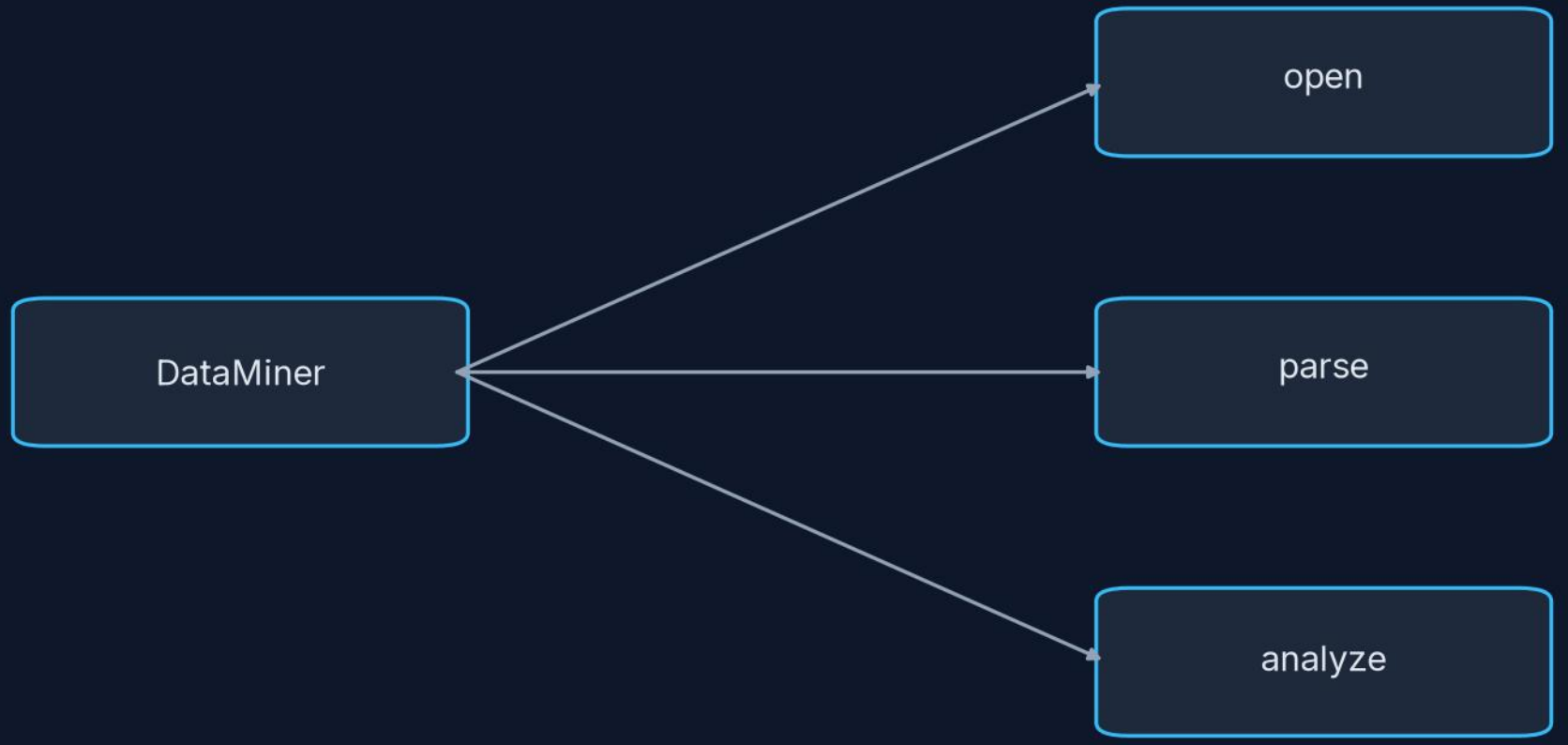
Today's Hook

- Today: Template Method as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Template Method structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Template Method
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/templatemethod/DataMiner.java
package com.buildwithwaqar.patterns.gof.behavioral.templatemethod;

import java.util.List;

public abstract class DataMiner {
    /** Template method — fixed algorithm skeleton. */
    public final String mine(String path) {
        var raw = open(path);
        var data = parse(raw);
        var report = analyze(data);
        close();
        return report;
    }

    protected abstract String open(String path);
    protected abstract List<String> parse(String raw);
    protected String analyze(List<String> data) { return "rows=" + data.size(); }
    protected void close() {}
}

final class CsvMiner extends DataMiner {
    protected String open(String path) { return "a,b\nc,d"; }
    protected List<String> parse(String raw) { return List.of(raw.split("\\n")); }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.creational.factorymethod;

public final class Demo {
    public static void main(String[] args) {
        System.out.println("=== Factory Method walkthrough ===");
        System.out.println(new EmailNotifier().notify("waqar@example.com", "Welcome"));
        System.out.println(new SmsNotifier().notify("+9198", "OTP 4421"));
        System.out.println("modern=" + NotificationChannel.PUSH.notify("device-9", "Build ready"));
        System.out.println("product=" + NotificationChannel.EMAIL.create().getClass().getSimpleName());
    }
}
```

Walkthrough Output

```
=== Template Method walkthrough ===
rows=2
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Template Method
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Template Method → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.